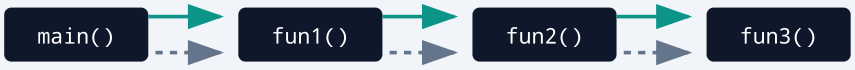
OBJECT-ORIENTED PROGRAMMING (OOP)

Fundamentals and Motivation

1. Functional Programming

In functional/procedural programming, a program is divided into functions that perform operations.

Function-call flow



After a function completes, control returns to the calling function.

Why OOP Became Popular

OOP was not invented simply because functional or procedural programs failed. OOP became popular because it offered a convenient way to organize large, complex software systems, especially systems that manage state.

2. Limitations of Functional/Procedural Programming

Consider a bank account program.

Initial approach: Global data and separate functions

```
int balance = 1000000;

void deposit()
{
    // Deposit operation
}

void withdraw()
{
    // Withdrawal operation
}

int main()
{
    deposit(50);
    withdraw(20);
}
```

Problem 2: Managing Multiple Objects Is Difficult

Suppose there are three bank accounts:

- Account 1
- Account 2
- Account 3

Without a suitable structure, separate variables may be required for each account, such as `balance1`, `balance2`, and `balance3`.

Arrays can help store multiple accounts' data, but the programmer must maintain the correct association between each account and its corresponding data.

Using Arrays

```
int balance[] = {500, 1000, 15000};

void deposit(int i, int amount)
{
    balance[i] += amount;
}
```

The function uses an index to identify which account's balance should be updated.

Problem 3: Adding New Features Increases Complexity

Consider a transfer operation:

```
void transfer(int from, int to, int amount)
{
    if (balance[from] >= amount)
    {
        balance[from] -= amount;
        balance[to] += amount;
    }
}
```

As an application grows, maintaining the relationship between data and the operations that belong to it becomes harder and more prone to errors, especially in large systems.

3. How OOP Addresses These Problems

OOP allows each **object** to keep its own data and its related operations together. For example, a bank account can contain:

- **Data:** Account holder's name and balance.
- **Operations:** Deposit, withdrawal, and potentially transfer.

Example: BankAccount Class

```
class BankAccount
{
public:
    string name;
    int balance;

    BankAccount(string n, int b)
    {
        name = n;
        balance = b;
    }

    void deposit(int amount)
    {
        balance += amount;
    }
};
```

Each `BankAccount` object has its own data and related operations.

Key idea: OOP organizes **data and the operations associated with that data** into objects.

4. Problem 4: Adding New Objects

Adding new objects in functional/procedural programming can require additional variables or additional bookkeeping.

Case 1: Without Arrays

For every new account, separate variables may be needed. For example, adding a third account could require:

```
string s3 = "Aadya";
int b3 = 1000;
```

This approach becomes inconvenient as the number of accounts increases.

Case 2: With Arrays

Arrays allow multiple accounts' data to be stored using indexed positions.

Conceptual example:

```
void addAccount(string name, int bal)
{
    names[cnt] = name;
    balance[cnt] = bal;
    cnt++;
}

int main()
{
    addAccount("Aadya", 1000);
}
```

- `names` stores account names.
- `balance` stores account balances.
- `cnt` identifies the next available index.
- `addAccount()` inserts the new account's data and increments the count.

Note: The handwritten notes do not clearly show the declarations and initializations of `names`, `balance`, or `cnt`. They must be supplied for a complete, compilable program.

Case 3: Using OOP

```
class BankAccount
{
public:
    string name;
    int balance;

    BankAccount(string n, int b)
    {
        name = n;
        balance = b;
    }
};

int main()
{
    BankAccount a3("Aadya", 1000);
}
```

A new account can be created by constructing a new `BankAccount` object and supplying its initial data.

Key Takeaway

- **Separate variables:** A new object may require more individual variables.
- **Arrays:** Multiple objects' data can be stored using indexes, but the relationships between parallel arrays must be maintained.
- **OOP:** Each object groups its own data and associated operations, making the program easier to organize and extend.

5. Summary: Problems and the OOP Approach

Problem in procedural programming	Explanation	How OOP helps
Global data accessibility	Many functions may access or modify the same data.	**Encapsulation** can control access to object data.
Managing multiple objects	Separate variables or carefully maintained indexes may be required.	Each object stores its own state.
Adding new features	Operations and data relationships become harder to maintain.	Related data and operations are grouped into **classes** and objects.
Adding new objects	More variables and bookkeeping may be needed.	New objects can be created from a class.

> **Important distinction:** Grouping data and functions into a class does not automatically guarantee data protection. In C++, access control such as `private` and `public` determines which parts of a class can be accessed directly.